Sprawozdanie z ćwiczenia nr 1

Predykaty geometryczne

Autor: Elżbieta Łabaj
Data: 29 października 2026

---

1 Dane techniczne

Specyfikacja środowiska obliczeniowego oraz parametrów technicznych:

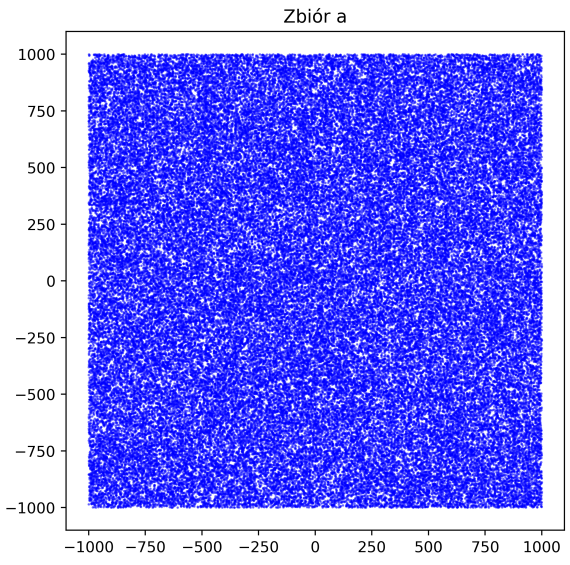
- **Język programowania:** Python 3.13.13
- **Procesor:** Intel(R) Core(TM) i5-8265U CPU 1.60GHz
- **System operacyjny:** Fedora Linux (x86_64)

2 Realizacja ćwiczenia

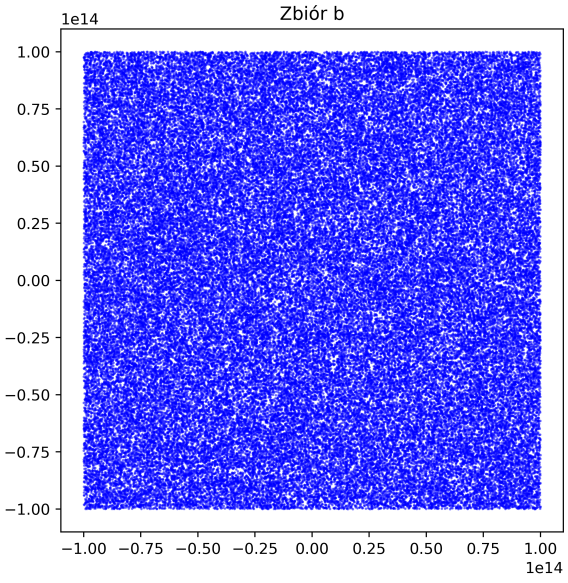
2.1 Generowanie zbiorów punktów:

Wygenerowano następujące zbiory punktów używając `np.random.uniform`:

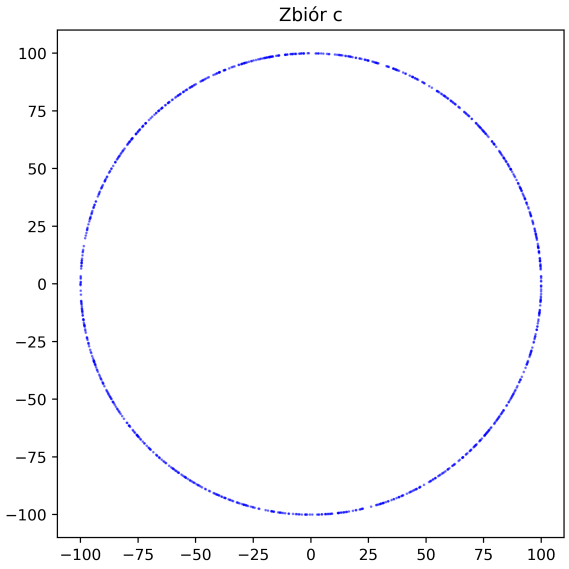
- **Zbiór a:** 10^5 losowych punktów z przedziału $[-1000, 1000] \times [-1000, 1000]$.
- **Zbiór b:** 10^5 losowych punktów z przedziału $[-10^{14}, 10^{14}] \times [-10^{14}, 10^{14}]$.
- **Zbiór c:** 1000 punktów na okręgu o środku $(0, 0)$ i promieniu $r = 100$, wygenerowanych z równania parametrycznego ($x = r\cos(2\pi t)$, $y = r\sin(2\pi t)$, $t \in [0, 1]$).
- **Zbiór d:** 1000 punktów leżących na prostej wyznaczonej przez wektor AB, gdzie $A = (-1.0, 0.0)$ oraz $B = (1.0, 0.1)$.



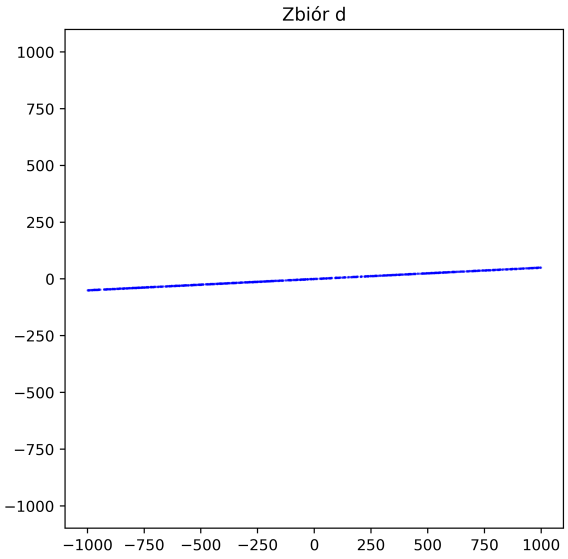
Rysunek 1: Zbiór a.



Rysunek 2: Zbiór b.



Rysunek 3: Zbiór c.



Rysunek 4: Zbiór d.

2.2 Klasyfikacja punktów względem prostej:

Każdy punkt c przydzielono do jednej z trzech kategorii względem prostej wyznaczanej przez wektor ab, gdzie $a = (-1.0, 0.0)$ oraz $b = (1.0, 0.1)$:

- na lewo od prostej ($\det > \epsilon$),
- na prawo od prostej ($\det < -\epsilon$),
- na prostej ($|\det| < \epsilon$).

Wyznacznik det liczono na cztery sposoby:

1. Wyznacznik macierzy (3×3). Użyto metody własnej (`cross3x3`) oraz `np.linalg.det` z biblioteki NumPy (`cross3x3_numpy`).

$$\det(a, b, c) = \begin{vmatrix} a_x & a_y & 1 \\ b_x & b_y & 1 \\ c_x & c_y & 1 \end{vmatrix} \tag{1}$$

2. Wyznacznik macierzy (2×2). Użyto metody własnej (`cross2x2`) oraz `np.linalg.det` z biblioteki NumPy (`cross2x2_numpy`).

$$\det(a, b, c) = \begin{vmatrix} a_x - c_x & a_y - c_y \\ b_x - c_x & b_y - c_y \end{vmatrix} \tag{2}$$

Użyto następujących wartości tolerancji dla zera (ϵ): $10^{-6}, 10^{-9}, 10^{-12}$ oraz następujących typów danych: `float32`, `float64`.

2.3 Analiza

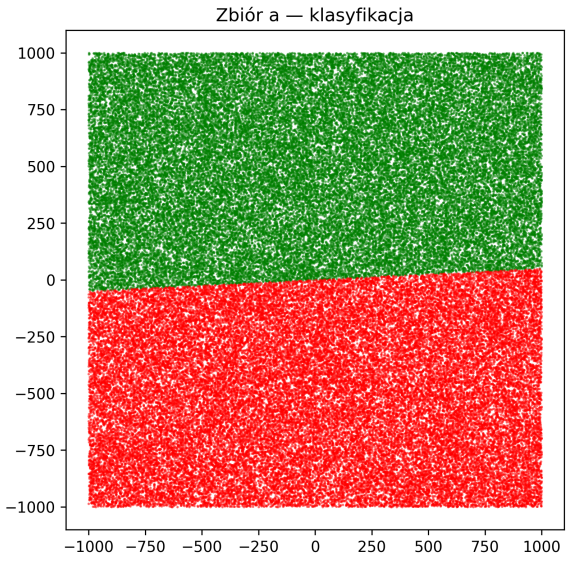
Analizowano poprawność oraz czas klasyfikacji punktów przez różne metody oraz dla różnych tolerancji i typów danych.

3 Wyniki

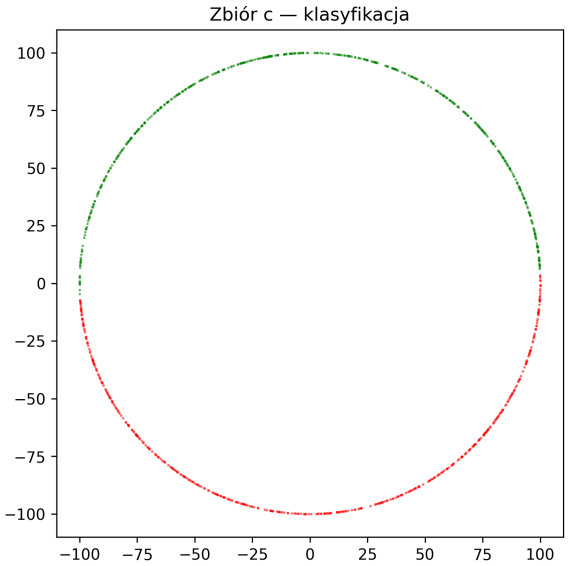
3.1 Zbiory a i c

Punkty w zbiorach a i c są równomiernie rozłożone, a ich współrzędne są z niewielkiego przedziału, dlatego dla obydwu zbiorów wszystkie metody sklasyfikowały wszystkie punkty poprawnie: 50242 punkty po lewej stronie, 49758 po prawej i 0 punktów na prostej.

Metody z biblioteki były wolniejsze od metod własnych, a liczenie wyznaczników 3×3 było wolniejsze od 2×2. Dokladne zestawienie pokazano w Tabeli 1.



Rysunek 5: Klasyfikacja punktów ze zbioru a.



Rysunek 6: Klasyfikacja punktów ze zbioru c.

Tabela 1: Zestawienie czasów wykonania metod dla zbiorów a i c.

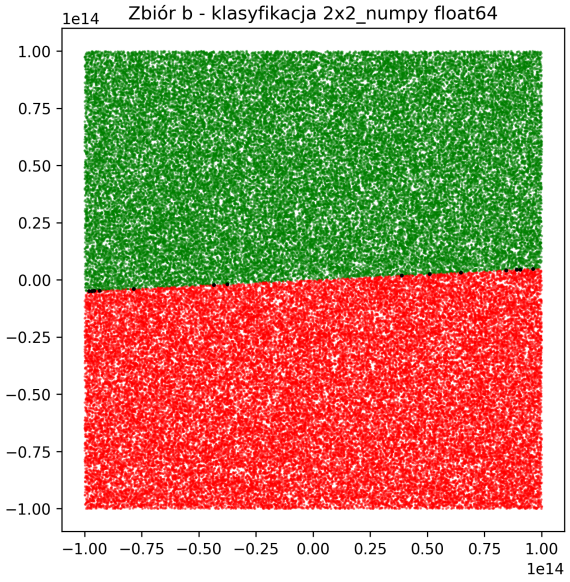
Zbiór	Typ danych	Epsilon	Metoda	Czas [s]
a	`float64`	10^{-12}	`3x3`	0.0056
a	`float64`	10^{-12}	`2x2`	0.0049
a	`float64`	10^{-12}	`3x3_numpy`	0.0295
a	`float64`	10^{-12}	`2x2_numpy`	0.0192
c	`float64`	10^{-12}	`3x3`	0.0001
c	`float64`	10^{-12}	`2x2`	0.0000
c	`float64`	10^{-12}	`3x3_numpy`	0.0003
c	`float64`	10^{-12}	`2x2_numpy`	0.0002

3.2 Zbiór b

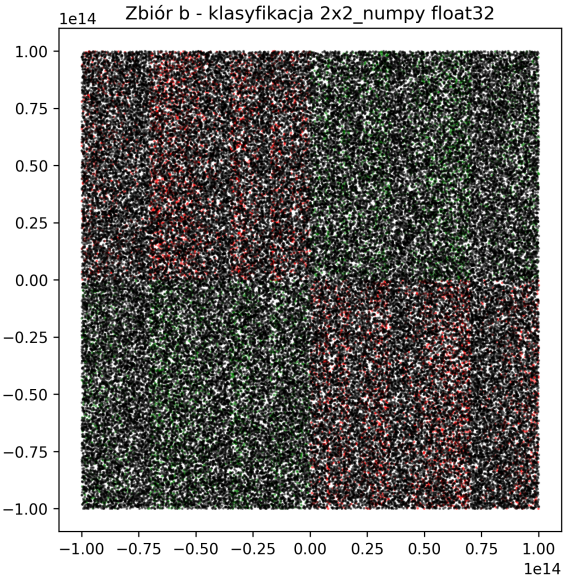
W zbiorze b współrzędne punktów były rzędu 10^{14} co spowodowało ogromny błąd przy klasyfikacji metodą wyznaczników macierzy 2×2. W tej metodzie liczone są najpierw różnice współrzędnych co zwłaszcza przy niskiej precyzji typu `float32` prowadzi do dużych błędów zaokrągleń i do fałszywej klasyfikacji punków jako współliniowych.

Na Rysunku 7 pokazano rozkład punktów przy użyciu typu `float64` oraz metody `cross2x2_numpy`, a na Rysunku 8 ten sam rozkład, ale przy użyciu typu `float32`. Na czarno zaznaczono punkty uznane za leżące na prostej.

W Tabeli 2 pokazano wyniki klasyfikacji również dla reszty metod. Wybór epsilona (tolerancji dla zera) nie wpływał na poprawność żadnej z metod.



Rysunek 7: Klasyfikacja punktów ze zbioru b metodą `2x2_numpy` na typie `float64`.



Rysunek 8: Klasyfikacja punktów ze zbioru b metodą `2x2_numpy` na typie `float32`.

Tabela 2: Zestawienie wyników klasyfikacji dla zbioru b.

Zbiór	Typ danych	Epsilon	Metoda	Czas [s]	Lewa	Prawa	Współliniowe
b	`float64`	10^{-12}	`3x3`	0.0063	50042	49958	0
b	`float64`	10^{-12}	`3x3_numpy`	0.0273	50042	49958	0
b	`float32`	10^{-12}	`3x3`	0.0026	50042	49958	0
b	`float32`	10^{-12}	`3x3_numpy`	0.0346	50042	49958	0
b	`float64`	10^{-12}	`2x2`	0.0032	50037	49951	12
b	`float64`	10^{-12}	`2x2_numpy`	0.0189	50036	49950	14
b	`float32`	10^{-12}	`2x2`	0.0018	0	0	100000
b	`float32`	10^{-12}	`2x2_numpy`	0.0157	6623	6618	86759

3.3 Zbiór d

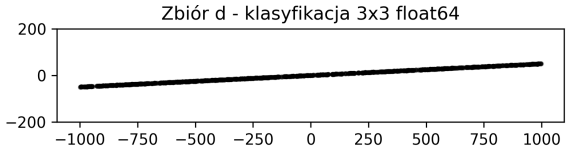
Wyniki klasyfikacji w zbiorze d okazały się najbardziej czułe na wybór wartości ϵ, typu danych oraz metody liczenia wyznacznika. Tak samo jak w zbiorze b, typ `float32` powodował bardzo duże błędy w obliczeniach z powodu zbyt niskiej precyzji.

Wszystkie punkty poprawnie zakwalifikowały tylko metody używające wyznacznika macierzy 3×3 na typie `float64`.

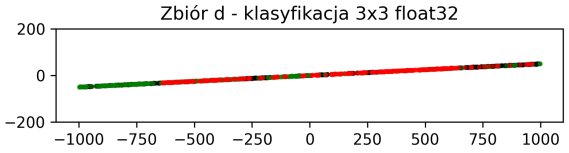
Dla wszystkich pozostałych metod zmniejszanie tolerancji dla zera powodowało zwiększanie ilości źle sklasyfikowanych punktów.

Na Rysunkach 9, 10, 11 i 12 porównano klasyfikacje punktów na prostej używając wyznaczników macierzy 2×2 i 3×3 oraz typów danych `float32` i `float64`.

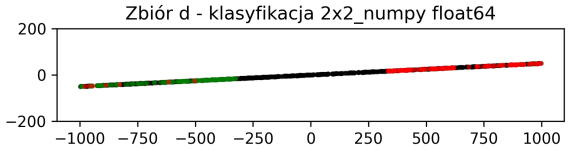
W Tabeli 3 przedstawiono wyniki dla wszystkich metod, typów danych i epsilonów.



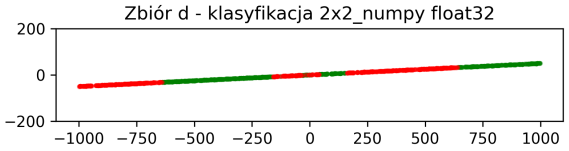
Rysunek 9: Klasyfikacja punktów ze zbioru d metodą `3x3` na typie `float64`.



Rysunek 10: Klasyfikacja punktów ze zbioru d metodą `3x3` na typie `float32`.



Rysunek 11: Klasyfikacja punktów ze zbioru d metodą `2x2_numpy` na typie `float64`.



Rysunek 12: Klasyfikacja punktów ze zbioru d metodą `2x2_numpy` na typie `float32`.

Tabela 3: Zestawienie wyników klasyfikacji dla zbioru d.

Zbiór	Typ danych	Epsilon	Metoda	Czas [s]	Lewa	Prawa	Współliniowe
d	`float64`	10^{-6}	`3x3`	0.0001	0	0	1000
d	`float64`	10^{-9}	`3x3`	0.0001	0	0	1000
d	`float64`	10^{-12}	`3x3`	0.0001	0	0	1000
d	`float64`	10^{-6}	`2x2`	0.0000	0	0	1000
d	`float64`	10^{-9}	`2x2`	0.0000	0	0	1000
d	`float64`	10^{-12}	`2x2`	0.0001	94	67	839
d	`float64`	10^{-6}	`3x3_numpy`	0.0003	0	0	1000
d	`float64`	10^{-9}	`3x3_numpy`	0.0003	0	0	1000
d	`float64`	10^{-12}	`3x3_numpy`	0.0003	0	0	1000
d	`float64`	10^{-6}	`2x2_numpy`	0.0002	0	0	1000
d	`float64`	10^{-9}	`2x2_numpy`	0.0002	0	0	1000
d	`float64`	10^{-12}	`2x2_numpy`	0.0002	118	110	772
d	`float32`	10^{-6}	`3x3`	0.0001	253	261	486
d	`float32`	10^{-9}	`3x3`	0.0000	314	304	382
d	`float32`	10^{-12}	`3x3`	0.0000	314	304	382
d	`float32`	10^{-6}	`2x2`	0.0000	152	148	700
d	`float32`	10^{-9}	`2x2`	0.0001	156	150	694
d	`float32`	10^{-12}	`2x2`	0.0000	156	150	694
d	`float32`	10^{-6}	`3x3_numpy`	0.0003	249	254	497
d	`float32`	10^{-9}	`3x3_numpy`	0.0003	407	410	183
d	`float32`	10^{-12}	`3x3_numpy`	0.0003	407	410	183
d	`float32`	10^{-6}	`2x2_numpy`	0.0003	487	483	30
d	`float32`	10^{-9}	`2x2_numpy`	0.0002	502	497	1
d	`float32`	10^{-12}	`2x2_numpy`	0.0002	502	498	0

4 Wnioski

Przeprowadzone eksperymenty można podsumować w następujących punktach:

1. **Wybór precyzji zmiennoprzecinkowej:** Wybór typu danych miał największy wpływ na poprawność wyników. Typ `float32` charakteryzujący się mniejszą precyzją niż `float64` prowadził do dużo większych błędów zaokrągleń i fałszywych klasyfikacji.

2. **Wpływ wielkości danych:** W zbiorze b, przy współrzędnych rzędu 10^{14} zastosowanie typu `float32` skutkowało drastyczną utratą cyfr znaczących w wyniku odejmowania bliskich sobie liczb, co doprowadziło do sytuacji, w której algorytm 2×2 błędnie zakwalifikował 100% punktów jako współliniowe. W zbiorach a i c, gdzie współrzędne były wielkości tylko 10^3 i 10^2 wszystkie metody działały całkowicie poprawnie.

3. **Metoda liczenia wyznacznika:** Metoda oparta na wyznaczniku stopnia 3 (3×3) okazała się znacznie stabilniejsza numerycznie niż metoda 2×2 , która wykazała dużo większą wrażliwość na błędy zaokrągleń.

4. **Wpływ progu tolerancji (ϵ):** Im niższa tolerancja dla zera tym na mniejszy margines błędu pozwalamy. Przez ograniczoną precyzję reprezentacji liczb w komputerze jednak wybranie zbyt restrykcyjnego progu powoduje utratę poprawności, co widać najlepiej przy zbiorze d (wszystkie punkty na prostej).

5. **Wydajność metod własnych oraz bibliotecznych:** Metody z biblioteki NumPy w niektórych zbiorach były dokładniejsze niż metody własne, a w innych mniej dokładne. W każdym zbiorze jednak metody własne były szybsze od tych bibliotecznych.